



MAILSHIELD

Email Header Analyzer

*A local, offline-first cybersecurity tool for detecting spoofing, phishing,
and authentication anomalies in email — project documentation*

Python • Flask • Jinja2 • HTML/CSS/JavaScript • Tesseract OCR • pytest
Version 1.0 — Local Cybersecurity Student Project

1. Project Overview

MailShield is a lightweight, local, offline-first web application that analyzes raw email headers — and, optionally, the message body or even a screenshot of an email — to surface indicators commonly associated with spoofing, phishing, and authentication failures. It is built as a classroom/portfolio-friendly demonstration of how much can be learned from an email's metadata alone, using entirely transparent, rule-based logic rather than a black-box model.

Why email headers matter

Every email carries a trail of metadata added by mail servers, mail clients, and (sometimes) attackers: the **From**, **Reply-To**, and **Return-Path** addresses, the **Received** chain each server adds as the message hops toward its destination, and authentication results (**SPF**, **DKIM**, **DMARC**) recorded by receiving mail servers. Careful analysis of these fields can reveal a sender address that doesn't match where replies actually go, a message that failed authentication, domains crafted to look like a trusted brand, or fabricated routing information — all before a human ever reads the message body.

Design principles

- **Offline-first:** core analysis never makes a network request. No DNS lookups, no IP/domain reputation services, no AI/LLM APIs.
- **Transparent scoring:** every point in the Suspicion Score is traceable to a named indicator with a configurable weight — never a black-box probability.
- **Conservative language:** the tool never declares a message "malicious" or "safe." It surfaces indicators worth reviewing and explains why, leaving the judgment call to the person reading the report.
- **Privacy by construction:** uploaded files and images are processed in memory and never written to disk or sent anywhere external.

2. How It Works

A single request flows through a pipeline of independent, composable stages. Each stage inspects a different aspect of the message and contributes zero or more *indicators* — small structured findings with a severity, a title, and a plain-language explanation of what happened, why it matters, and what to check next.

Stage	Module	What it does
Header parsing	parser.py	Extracts From/To/Subject/Date/Message-ID/Received etc. using Python's built-in email module; flags malformed lines, duplicate headers, and bracket mismatches.
Authentication	authentication.py	Reads SPF/DKIM/DMARC results exactly as reported by the headers, plus the DKIM signing domain and selector. No live DNS or cryptographic re-verification is performed.

Stage	Module	What it does
Sender & domain checks	indicators.py	Compares From/Reply-To/Return-Path/DKIM domains; detects lookalike domains (character substitution), brand impersonation, excessive subdomains, and display-name spoofing (e.g. "PayPal" from a Gmail address).
Routing / IP analysis	routing.py	Parses the Received chain into chronological hops and classifies each IP as Public, Private, Loopback, Reserved, Documentation, or Invalid.
Body & link analysis	content.py	Parses the message body (if present): link display-text-vs-destination mismatches, IP-literal or shortened links, the "user@host" URL trick, punycode domains, and phrase-based detection for urgency, financial/prize, credential-request, legal-threat, and job-scam language.
Image scan (OCR)	ocr.py	Recognizes text from an uploaded screenshot using the local Tesseract engine, then runs the same phrase/link heuristics against the recognized text. A strictly more limited analysis, since a screenshot never exposes SPF/DKIM/DMARC or a link's real destination.
Risk scoring	risk_engine.py	Sums a configurable per-indicator weight into a 0-100 Suspicion Score and maps it to LOW / MODERATE / HIGH / CRITICAL, with a severity floor so a single strong finding is never undersold.
Reporting	app.py / templates	Renders the dashboard, JSON API response, and exportable TXT/JSON report.

3. Technology Stack

The project deliberately avoids heavy frameworks, databases, and cloud/AI services. Everything below either ships with Python or is a small, well-known library installed via `pip`.

Language & runtime

Component	Role
Python 3.11+	Core implementation language for the entire backend and analysis engine.
HTML5	Structure of every page, rendered server-side via Jinja2 templates.
CSS3 (hand-written)	Dark cybersecurity dashboard styling. No Bootstrap/Tailwind or other CSS framework.
Vanilla JavaScript	Sample-loading dropdown, Clear button, and client-side file validation. No React/Vue/Angular.

Third-party Python packages

Package	Version	Purpose
Flask	3.1.x	Web framework: routing, request/response handling, Jinja2 template rendering, session cookies for CSRF tokens.
Werkzeug	3.1.x	WSGI toolkit Flask is built on (request parsing, dev server, the 413 "payload too large" exception).
Jinja2	3.1.x	Server-side HTML templating (installed automatically with Flask).
pytesseract	0.3.x	Thin Python wrapper that shells out to the local Tesseract OCR binary and returns recognized text.
Pillow (PIL)	12.x	Opens and decodes the uploaded image file before handing pixel data to Tesseract.
pytest	9.x	Automated test runner used for all 76 unit/integration tests (development dependency, not required to run the app).

External system dependency

Tesseract OCR — an open-source optical character recognition engine (installed separately, e.g. via `winget/brew/apt`, not `pip`). It runs entirely on the local machine; no image data is ever sent anywhere. This is the only feature (image screenshot scanning) that requires it — every other feature works with just the Python packages above.

Python standard library modules used

`email` (RFC 822 parsing), `re` (pattern matching throughout the heuristics), `ipaddress` (IP classification), `datetime` (timestamp consistency checks), `urllib.parse` (URL/hostname parsing), `html.parser` (safe HTML tag stripping/link extraction, never executes markup), `json`, `io`, `os`, `secrets` (CSRF tokens), and `shutil`.

Explicitly NOT used

No database, no Docker, no Redis, no React/Vue/Angular/Electron, no AI/LLM API, no cloud reputation service, no build step / bundler. This keeps the project small, auditable, and runnable entirely offline on a low-resource machine.

4. Feature Summary

- Paste raw headers, upload a **.eml/.txt** file (1 MB limit), or upload a **screenshot** of an email (.png/.jpg/.webp/.gif/.bmp, 5 MB limit).
- SPF / DKIM / DMARC result extraction, reported exactly as the headers present them.
- Sender identity checks: From vs. Reply-To vs. Return-Path vs. DKIM signing domain.
- Domain heuristics: excessive subdomains, character-substitution lookalikes (paypa1.com), and brand-name impersonation (secure-paypal-login.example).
- Display-name impersonation detection — catches a scam sent through a fully authenticated free-webmail account (e.g. "PayPal Support" from a real Gmail address).
- IP address classification: Public / Private / Loopback / Reserved / Documentation / Multicast / Invalid.
- Full Received: chain parsed into chronological order with timestamp-consistency checks.
- Body/link analysis: text-vs-destination mismatches, raw-IP or shortened links, the "user@host" URL trick, punycode domains, and phrase detection for urgency, financial/prize, credential-request, legal-threat/extortion, and job-scam language.
- Attachment filename check (dangerous extensions, double-extension disguises) — metadata only, contents are never opened.
- Local OCR image scanning for screenshots of emails.
- Transparent, configurable 0-100 Suspicion Score with LOW/MODERATE/HIGH/CRITICAL risk levels.
- A "What happened? / Why does it matter? / What should you check?" explanation for every indicator.
- Three built-in sample emails (safe / suspicious / spoofed) for instant demonstration.
- JSON API at POST /api/analyze for programmatic use.
- Export findings as a .txt or .json report.

5. Installation

Requires Python 3.11+.

```
python -m venv .venv

# Windows (PowerShell)
.venv\Scripts\Activate.ps1

# macOS / Linux
source .venv/bin/activate

pip install -r requirements.txt
python app.py
```

Then open <http://127.0.0.1:5000> in a browser.

Optional: enabling image scanning

Every feature except screenshot scanning works with the steps above. To also scan images, install the Tesseract OCR engine separately:

```
# Windows
winget install --id UB-Mannheim.TesseractOCR -e

# macOS
brew install tesseract

# Debian / Ubuntu
sudo apt install tesseract-ocr
```

6. Usage Guide

- 1 Open the app in a browser at `http://127.0.0.1:5000`.
- 2 Paste raw email headers, upload a `.eml/.txt` file, upload an email screenshot, or click **Load Sample** to try a built-in example.
- 3 Click **Analyze Headers**.
- 4 Review the Suspicion Score, authentication results, and indicator list — each indicator expands to show what happened, why it matters, and what to check next.
- 5 Click **Export Report** to save the findings as a `.txt` or `.json` file.

Programmatic use (JSON API)

```
POST /api/analyze
Content-Type: application/json

{"headers": "From: example@example.com\n..."}
```

Returns a JSON object with `score`, `risk_level`, `authentication`, `sender`, `domain_analysis`, `content_analysis`, and a list of indicators.

7. Understanding the Suspicion Score

The score is a transparent sum of per-indicator weights (defined in `analyzer/risk_engine.py`), capped at 100 — never a claimed probability of phishing.

Score range	Risk level	Meaning
0-19	LOW	No major indicators, or only very minor ones.
20-39	MODERATE	A handful of indicators; worth a second look.
40-69	HIGH	Multiple indicators, or one strong specific finding.
70-100	CRITICAL	Numerous or severe indicators stacking together.

Severity floor: a single HIGH-severity indicator (e.g. an explicit credential request, or a law-enforcement extortion threat) always floors the overall risk level at HIGH, even if few other indicators happened to co-occur. This matters most for image-based scans, which have structurally fewer simultaneous signals available than full header analysis.

How to read a mid-range score: look at *which* indicators fired, not just the count. A HIGH-severity finding like a failed DMARC check or a confirmed link mismatch is strong evidence. A score built entirely from LOW-severity findings (a missing Message-ID, a keyword in a domain) is much weaker evidence and common in legitimate bulk mail. The indicator list and its explanations are the actual evidence — the score is a triage number, not a verdict.

8. Testing & Quality Assurance

The project ships with an automated pytest suite (76 tests at the time of writing) covering header parsing, SPF/DKIM/DMARC extraction, body/link heuristics, OCR text analysis, risk scoring, and Flask routes end-to-end.

```
pytest
```

Areas covered:

- **test_parser.py** — valid/missing/duplicate/malformed headers, size limits.
- **test_authentication.py** — SPF/DKIM/DMARC pass/fail/none, conflicting results.
- **test_content.py** — link mismatches, URL obfuscation, phrase-based scam categories, attachment filename checks.
- **test_ocr.py** — sender/subject extraction from recognized text, and a real end-to-end OCR run against the Tesseract engine.
- **test_risk_engine.py** — score calculation, risk-level thresholds, the severity floor.
- **test_routes.py** — the Flask app end-to-end, including CSRF protection and the image-upload code path.

9. Security & Privacy

- The development server binds to 127.0.0.1 only, by default.
- Core analysis is 100% offline: no DNS lookups, no IP/domain reputation services, no outbound network requests of any kind.
- The app never sends email, executes attachments, or opens URLs automatically.
- Uploaded files are validated by extension and size, processed in memory, and never written to disk.
- An uploaded screenshot is never fetched, forwarded, or sent to any external service — OCR runs locally via Tesseract only.
- The message body is parsed only for the static text/link checks described above — never displayed in full and never used to fetch or visit a link.
- Jinja2 auto-escaping is used throughout, so header/body content is never rendered as executable HTML.
- The analyze/export forms are protected by a per-session CSRF token.
- Errors are shown as friendly messages; stack traces are never exposed to the browser.

10. Limitations

Header and body analysis together still cannot guarantee that an email is malicious or safe. SPF/DKIM/DMARC results are reported exactly as the headers present them; MailShield does not perform live DNS lookups or cryptographic signature verification, and does not check IP/domain reputation. The body/link checks are static text/HTML parsing against a small, conservative brand/keyword list — they deliberately do not attempt grammar or writing-quality scoring, which reliably requires NLP/ML and is out of scope for this offline tool.

Image scans are more limited still: OCR accuracy depends on image quality, and a screenshot never exposes SPF/DKIM/DMARC, the Received chain, or a link's real underlying destination — only what was visibly rendered.

Every indicator is a signal worth reviewing, not a verdict. Always corroborate findings with independent verification before taking action on a message.

11. Project Structure

```
mailshield/
|-- app.py Flask routes, CSRF, export/report generation
|-- requirements.txt
|-- analyzer/
| |-- parser.py Header parsing + line-level anomaly checks
| |-- authentication.py SPF/DKIM/DMARC extraction
| |-- routing.py Received: chain parsing
| |-- indicators.py Header-based indicator/heuristic detection
| |-- content.py Body/link analysis
| |-- ocr.py Image-screenshot analysis (local OCR)
| |-- risk_engine.py Configurable scoring + risk level
| `-- utils.py Domain/IP/date/URL helpers
|-- templates/ Jinja2 templates (dark dashboard UI)
|-- static/ CSS + vanilla JS
|-- samples/ Built-in demo emails
`-- tests/ pytest suite (76 tests)
```

MailShield is intended as a local, educational cybersecurity tool. It does not replace a full email security gateway, and its findings should always be treated as leads for further investigation rather than final conclusions.